

操作系统

何柯

2026 年 1 月 3 日

目录

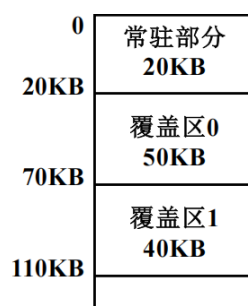
1	内存管理	3
1.1	基础概念	3
1.2	内存分配方案	4
1.2.1	连续分配	4
1.2.2	离散分配	6
1.3	虚拟内存技术	9
1.4	优化技术	11
2	文件系统	12
2.1	文件系统基础	12
2.2	文件组织方式	13
2.2.1	空闲空间管理	13
2.2.2	接口设计	15
2.2.3	连续分配	16
2.2.4	链接分配	17
2.2.5	索引分配	18
2.3	文件目录与路径	19
2.3.1	文件链接	19
2.4	文件操作接口	20
2.5	典型文件系统	20

2.6	文件系统性能	21
2.7	虚拟文件系统 (VFS)	21
3	设备管理	22
3.1	基本概念	22
3.2	硬盘	25
3.2.1	机械硬盘	25
3.2.2	固态硬盘	25
3.2.3	磁盘调度算法	27
3.2.4	分区与格式化	30
3.3	时钟	30
3.4	I/O 系统层次架构	31
3.5	中断处理程序	32
3.5.1	上下文保护与恢复	32
3.5.2	上半部与下半部机制	33
3.6	设备独立内核软件技术	34
3.6.1	缓冲技术	35
3.6.2	异步 I/O (AIO)	37
3.6.3	异步 I/O (AIO)	37
3.6.4	SPOOLing 技术 (假脱机)	39
3.6.5	设备分配与回收	40

1 内存管理

1.1 基础概念

- 逻辑地址：这是程序视角下的地址，也称为相对地址，程序通常认为自己是地址 0 开始存放的
- 物理地址：这是硬件实现视角下的地址，也就是数据在内存条上实际存储的绝对位置，如果程序的基址（起始位置）是 1000，逻辑地址 500 对应的物理地址实际上是 1500
- 重定位技术：将程序的逻辑地址转换为物理地址的技术
 - 静态重定位 (**Static Relocation**)：编译器或加载器直接将程序中的指令地址修改为绝对物理地址，由软件完成。缺点是程序一旦加载到内存就不能移动，不支持多道程序运行。
 - 动态重定位 (**Dynamic Relocation**)：在程序运行时进行地址转换，依赖于重定位寄存器（基址寄存器）的硬件支持。计算公式：物理地址 = 逻辑地址 + 基址寄存器的值。优点是程序可以加载到内存的任意位置，运行时可以在内存中移动。
- 内存保护 (Memory Protection)：使用界限寄存器 (Limit Register) 防止越界访问。检查条件为：逻辑地址 < 界限寄存器值。若不满足则触发越界异常；若满足则加上基址寄存器的值形成物理地址。
- 覆盖与交换技术
 1. 覆盖技术 (Overlay)
 - (a) 核心场景：针对单个程序。当程序代码量大于物理内存时的解决方案
 - (b) 基本原理：程序员手动切分程序为若干功能模块。常驻区存放核心代码并始终在内存中；覆盖区存放互斥模块（不同时执行），共用同一内存区域。



- (c) 例子：程序总大小 190KB，内存 110KB。常驻区放 A 模块 (20KB)；覆盖区 0 放 B(50KB) 和 C(30KB) 互斥；覆盖区 1 放 D、E、F 中最大的 E(40KB)；总需求 $20+50+40 = 110\text{KB}$ 。
- (d) 优点：解决大程序在小内存上运行
- (e) 缺点：程序员必须手动规划模块调用关系，难度大

2. 交换技术 (Swapping)

- (a) 核心场景：针对多个程序（多道程序环境）。当内存里挤满了进程，新的进程进不来，或者被阻塞的进程占着茅坑不拉屎时，怎么办
- (b) 基本原理：操作系统充当”调度员”，把暂时不用的进程整个搬到硬盘（外存）里，腾出地方给急需运行的进程。换出 (Swap Out) 是指进程从内存移到硬盘对换区；换入 (Swap In) 是指进程从硬盘搬回内存运行。
- (c) 特点：透明性强，支持多道程序，整体交换
- (d) 缺点：硬盘 I/O 开销大，性能较低

1.2 内存分配方案

1.2.1 连续分配

1. 固定分区分配 (Fixed Partitioning):

- 原理：将内存划分为若干固定大小的分区，每个分区可以容纳一个进程
- 优点：实现简单，易于管理

- 缺点：内存利用率低，可能产生大量碎片；数量固定；大小死板

2. 动态分区分配 (Dynamic Partitioning):

按程序需求分配不预先划分分区大小，存储结构有空闲分区表和空闲分区链表两种示意图如下：



分区号	大小	起始地址
1	8KB	24KB
2	12KB	128KB
3	8KB	248KB
4	...	...
5	...	...

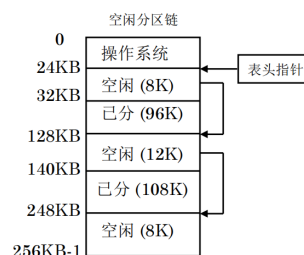


图 1: 内存布局图

图 2: 空闲分区表

图 3: 空闲分区链

但是动态分区方案依然存在一个问题那就是外部碎片问题 (外部碎片是指内存中存在许多小的空闲分区，这些分区的总量虽然足以满足一个新作业的申请要求，但由于它们在物理位置上是不连续的 (被已分配的作业隔开了)，导致无法将该作业装入内存) 有如下两种解决方案:

(a) 紧凑化 (Compaction): 移动所有进程-> 开销极大

(b) 分配算法优化:

- 首次适应算法 (First Fit): 从头开始查找第一个满足要求的空闲分区 (特点: 优先利用内存低地址端，高地址端有大空闲区。但低地址端有许多小空闲分区时会增加查找开销)
- 循环首次适应算法 (Next Fit): 从上次分配结束的位置开始查找 (特点: 使存储空间的利用更加均衡，但会使系统缺乏大的空闲分区)
- 最佳适应算法 (Best Fit): 查找所有满足要求的空闲分区，选择最小的一个 (特点: 减少内存浪费，但会产生大量小的不可用碎片)

- 最差适应算法 (Worst Fit): 查找所有满足要求的空闲分区, 选择最大的一个 (特点: 剩下的空闲区比较大, 但当大作业到来时, 其存储空间的申请往往得不到满足)

对于算法的衡量好坏的标准: 对于某一个作业序列来说, 若某种分配算法能将该作业序列中所有作业安置完毕, 则称该分配算法对这一作业序列合适, 否则称为不合适

3. 伙伴系统 (Buddy System): 采用伙伴算法对空闲内存进行管理该方法通过不断对分大的空闲存储块来获得小的空闲存储块。当内存块释放时, 应尽可能合并空闲块。基本原理:

- 内存块大小均为 2^k 的形式
- 分配时, 找到最小的满足需求的 2^k 块
- 若找到的块大于需求, 则不断对半拆分, 直到接近需求大小
- 释放时, 检查相邻伙伴块 (大小相同) 是否空闲, 若是则合并

优缺点:

- 优点: 回收效率高: 合并算法简单快速; 灵活性: 比固定分区更灵活, 且避免了动态分区中复杂的“紧凑”操作
- 缺点: 内部碎片: 由于分配块必须是 2 的幂次, 若申请 65KB 需分配 128KB, 会产生明显的内存浪费; 拆分/合并开销: 频繁的分配与回收会导致多次拆分与合并操作

例子: 假设初始内存为 1MB, 作业申请序列为 A(200K)、B(120K)。分配 A(200K) 时分配 256K 块, 剩余 768K; 分配 B(120K) 时分配 128K 块, 剩余 640K; 释放 A 时释放 256K 块, 检查伙伴后无法合并。

1.2.2 离散分配

1. 分页管理 (Paging):

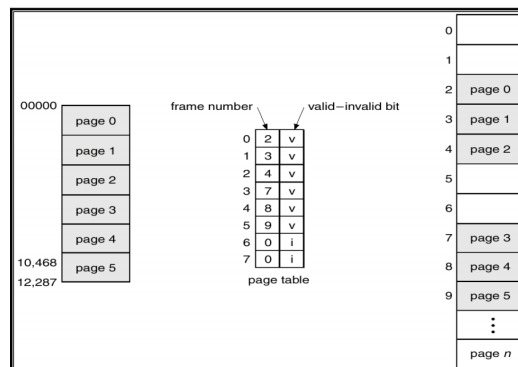


图 4: 分页管理示意图

页表 (如图 4): 逻辑页面-> 物理块页面-> 页框 (就是将进程划分为若干个固定大小的页, 每页大小通常为 4KB, 然后将这些页映射到物理内存中的页框)

计算地址:

$$\text{物理地址} = \text{页框号} \times \text{页大小} + \text{页内偏移}$$

快表: 并行查找缓存最近使用的页表项, 提高地址转换速度

表 1: 页表与快表比较

特性	页表 (Page Table)	快表 (TLB)
位置	主内存 (RAM)	CPU 内部 (MMU)
介质	DRAM (普通存储器)	SRAM (联想存储器)
容量	大, 存放全部页表项	小, 通常只有 64-1024 项
访问速度	慢	极快
查找方式	索引查找	并行比较查找

有效时间计算: 给出一个例子吧: 假设内存一次存取时间是 m , 联想寄存器的查找时间是 n , 命中率为 p 那么, 有效存取时间 EAT 为:

$$EAT = (n + m) \times p + (n + 2m) \times (1 - p) = n + m + (1 - p) \times m$$

解释: 命中时需要查找快表和访问内存, 未命中时需要查找快表、访问页表和访问内存两次

不同的分页管理方式：

- 多级页表：将页表划分为多级结构，减少内存占用

例子：假设逻辑地址为 32 位，页面大小为 4KB (2 的 12 次方)，则页内偏移占 12 位，剩余 20 位用于页号。

页面大小 $\rightarrow 4KB = 2^{12}$ bytes $\rightarrow$ 页内偏移 12 位 $\rightarrow$ 剩余 20 位用于页号

将页号划分为两级：高 10 位作为一级页表索引，低 10 位作为二级页表索引 (因为每个页表项长度是 4 字节，所以每个页表可以包含 2^{10} 个页表项)

- 哈希页表：使用哈希函数映射逻辑页号到页表项，适用于大地址空间 (哈希函数 $\rightarrow$ 逻辑页号 (哈希链表中顺序查找) $\rightarrow$ 物理块号)

例子：假设逻辑页号为 **12345**，哈希表大小为 **1024**，哈希函数为 $h(x) = x \bmod 1024$ ，则 $h(12345) = 12345 \bmod 1024 = 57$ ，查找哈希表索引 **57** 处的链表以找到对应的页表项。57 $\rightarrow$ (12345, 5) $\rightarrow$ (6789, 12) $\rightarrow$...

- 反向页表：直接按序查找物理块对应的逻辑页，节省内存

2. 分段管理 (Segmentation): 段是逻辑划分的结构，按程序需求分配不预先划分分区大小，存储结构有段表，每个段有段号、段基址、段界限等信息

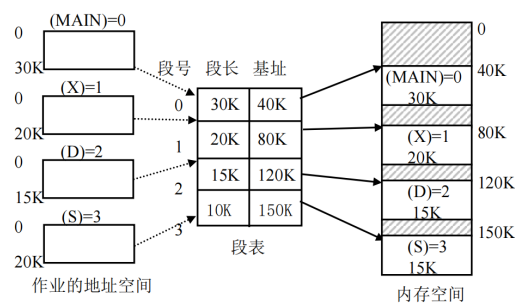


图 5: 分段管理示意图

3. 段页式管理 (Segmented Paging): 就是先用分段管理将程序划分为若干段，然后对每个段再进行分页管理

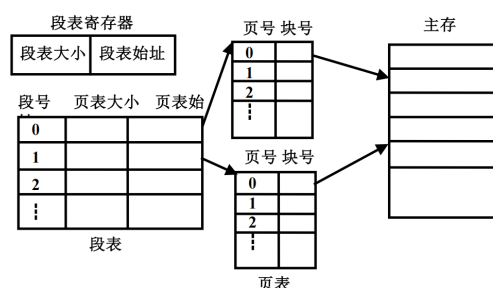


图 6: 段页式管理示意图

1.3 虚拟内存技术

虚拟内存技术: 将用户逻辑内存与物理内存分开, 使得程序可以使用比实际物理内存更大的地址空间

局部性原理: 程序在执行过程中, 在较短的一段时间内, 所执行的指令地址和操作数地址往往集中在一定的存储区域内

- 时间局部性: 最近访问过的数据和指令在不久的将来可能会被再次访问 (循环结构 → 带来时间局部性。)
- 空间局部性: 与最近访问过的数据和指令地址相近的数据和指令在不久的将来可能会被访问 (顺序执行 → 带来空间局部性。)

虚拟存储器的实现方法:

- 请求分页 (Demand Paging): 只有在页面被访问时才加载到内存
 - (a) 扩充后的页表结构:
 - 有效位 (Valid Bit): 指示该页是否在内存中
 - 修改位 (Dirty Bit): 指示该页是否被修改过
 - 访问位 (Accessed Bit): 指示该页是否被访问过
 - 页号和物理块号
 - 外存位置指针: 指向该页在外存中的位置
 - (b) 页错误:
 - i. 发生条件: 访问的页不在内存中 (有效位为 0)

ii. 处理步骤:

- A. 检查页表以确定引用是否合法
- B. 引用非法则终止, 有效但不在内存则调入
- C. 找到一个空闲帧
- D. 把页换入页框
- E. 修改页表, 使其有
- F. 重启指令

(c) 缺页中断: 就是处理页错误的过程

(d) 有效访问时间 (EAT) 计算: 内存的读写周期为 t , 缺页中断服务时间为 t_1 (包含读入缺页、页表更新、快表更新时间) 快表的命中率为 α , 缺页中断率为 f , 快表访问时间为 ε , 则有效存取时间可表示为

$$EAT = \alpha \times (t + \varepsilon) + (1 - \alpha) \times [(1 - f) \times 2(t + \varepsilon) + f \times (t_1 + 2(t + \varepsilon))]$$

为什么是 $2(t + \varepsilon)$ 呢? 因为未命中快表时, 需要先访问页表 (一次内存访问), 然后再访问实际数据页 (第二次内存访问), 对于快表则是访问 + 修改

(e) 页面置换算法:

- FIFO: 先入先出, 缺点: 可能会淘汰掉经常使用的页面
- OPT: 最佳置换, 缺点: 需要预知未来访问情况, 实际难以实现
- LRU: 最近最少使用, 优点: 较好地利用了局部性原理, 缺点: 实现复杂, 需要维护访问时间信息
- LRU 近似算法: 使用访问位, 定期清除访问位, 选择未被访问的页面进行置换
- CLOCK: 时钟算法, 使用一个指针循环扫描页面, 选择访问位为 0 的页面进行置换
- 改进时钟算法: 结合修改位, 优先选择访问位和修改位都为 0 的页面进行置换
- LFU: 最不经常使用, 选择访问次数最少的页面进行置换

(f) 页面分配算法

- i. 平均分配：将物理内存均匀分配给所有进程
- ii. 按比例分配：根据进程大小按比例分配内存
- iii. 按优先级分配：分为两部分，先按比例分配，再根据进程优先级分配内存

(g) 抖动 (Thrashing):

如果运行进程的大部分时间都用于页面的换入/换出，而几乎不能完成任何有效的工作，则称此进程处于抖动状态。抖动又称为颠簸、颤动。

产生原因：

- 进程分配的物理块太少
- 置换算法选择不当
- 全局置换使抖动传播

(h) 工作集模型 (Working Set Model):

- 定义：在某一时间段内，进程实际使用的页面集合
- 工作集窗口 (Δ)：定义为最近 Δ 时间内访问的页面集合
- 工作集大小 (WSS)：工作集窗口内的页面数量
- 目标：保持进程的工作集在内存中，减少缺页率

举个例子：观察窗口 (Δ)：管理员盯着你看了 10 分钟（这就是工作集窗口）；工作集 (WSS)：他发现这 10 分钟里，你反反复复翻阅的其实只有《数学》、《物理》、《化学》这 3 本书决策：虽然你有 100 本参考书，但在当前阶段，只要给你书桌上腾出 3 个位置，你就能顺畅工作了

- 请求分段 (Demand Segmentation):

只有在段被访问时才加载到内存。请求分段的主要优势包括：有利于动态增长、允许按段进行编译、有利于共享和保护。

1.4 优化技术

写时复制 (**COW**)：只要没人要修改数据，大家就共享同一份；一旦有人要修改，再给他单独复制一份

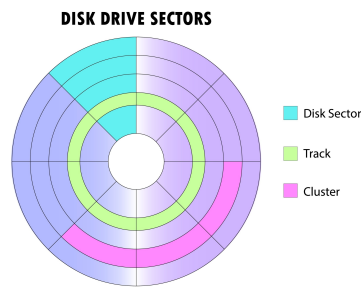


图 7: 扇区示意图

这个过程利用了“只读”标记和“欺骗”手段：

1. 初始状态：父子进程共享同一页，页表项标记为“只读”
2. 修改请求：当某个进程尝试写入该页时，CPU 发现该页面是“只读”的，于是报错（触发异常/中断）
3. 处理异常：操作系统捕获这个中断，发现是因为“写时拷贝”机制引起的，于是它说：“好吧，既然你要改，那我现在给你复制一份。
4. 结果：现在，进程 1 拥有了自己的新页面，进程 2 还在用旧页面。只有被修改的那一页发生了复制，其他没改的页依然共享。

2 文件系统

2.1 文件系统基础

- 扇区：磁盘上最小的存储单位，通常为 512 字节磁盘的盘片被划分成许多同心圆（磁道），每个磁道又被切分成许多小弧段，这些小弧段就是“扇区”
- 元数据：即描述文件属性的信息，而不是文件内容本身，元数据通常包含以下内容：
 1. 标识 (文件名/类型/Inode)
 2. 位置 (文件在磁盘上的存储地址)

3. 属性 (权限/所有者/创建时间/修改时间/大小)
 4. 时间
 5. Inode: 一种数据结构, 用于存储文件的元数据和磁盘块地址 (固定 256 字节)
 6. Dient: 目录项, 包含文件名和对应的 Inode 号
- 视图转换: 视图转换操作主要依靠 **Inode** (提供静态索引地图) 和 **bmap** (提供动态查找算法) 进行实现。其目的是将用户眼中的“连续字节流 (逻辑视图)”转换为磁盘眼中的“离散块 (物理视图)”。
- 工作链条: 文件名 $\xrightarrow{\text{查目录 (dient)}}$ Inode 号 $\xrightarrow{\text{查 Inode 表}}$ Inode 结构体 $\xrightarrow{\text{查 Addr 数组}}$ 物理块号 $\xrightarrow{\text{读磁盘}}$ 文件数据
- 加入 **bmap** 后 (细化逻辑块到物理块的映射): 在多级索引结构中, 简单的“查 Addr 数组”被扩展为由 **bmap** 函数执行的复杂计算过程:
1. 逻辑坐标计算: 系统根据用户提供的字节偏移量算出逻辑块号 (LBN)。逻辑块号 (LBN) = $\lfloor \text{字节偏移量} / \text{块大小} \rfloor$
 2. **bmap** 映射逻辑: 内核调用 **bmap(inode, LBN)**, 根据 LBN 的大小在 Inode 的混合索引表中顺藤摸瓜:
 - 直接寻址: 若 $LBN < 12$, 则 物理块号 (PBN) = $\text{Inode.Addr}[LBN]$ 。
 - 一次间接: 若 $12 \leq LBN < 1036$, 则 **bmap** 读取 $\text{Inode.Addr}[12]$ 指向的索引块并查找。
 - 多次间接: 若 LBN 更大, **bmap** 需逐级访问二级或三级间接块指针 ($\text{Inode.Addr}[13]$ 或 $[14]$), 最终锁定 PBN。
 3. 最终物理寻址: $(\text{Inode} + \text{LBN}) \xrightarrow{\text{bmap 算法}}$ 物理块号 (PBN) $\xrightarrow{\text{驱动读取}}$ 物理扇区数据

2.2 文件组织方式

2.2.1 空闲空间管理

1. 位图法 (Bitmap Method):

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	1	1	0	0	1	1	0	1	1	1	0	1	1	1	1	1
1	0	0	0	0	1	1	1	1	1	0	0	0	0	0	0	1
2	1	1	1	1	1	1	0	1	1	1	1	0	0	0	0	0
3																
4	...															
⋮																

图 8: 位图法示意图

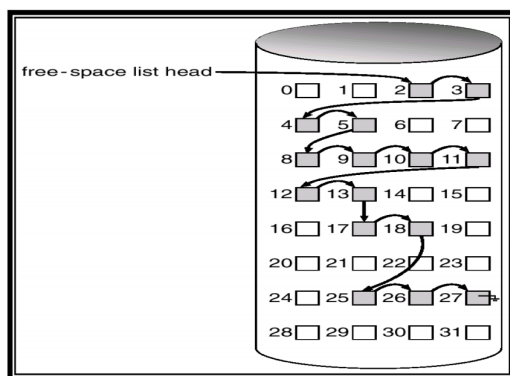


图 9: 空闲链表法示意图

用一个二进制位（0 或 1）代表一个物理块的状态。0 表示空闲，1 表示已分配。

- 优点：容易找到连续的空闲块。
- 缺点：当磁盘很大时，位图本身会占用大量内存。

2. 空闲链表法 (Linked List Method):

把所有空闲块连成一串链表，每个空闲块包含一个指向下一个空闲块的指针。

- 优点：分配简单。
- 缺点：如果要申请大量块，需要沿着链表一个一个找，效率极低。

3. 成组链接法 (Grouped Link Method):

将所有空闲块按组划分，每一组的第一个块会记录下一组所有空闲块的块号建立了一种“层层递进”的结构。

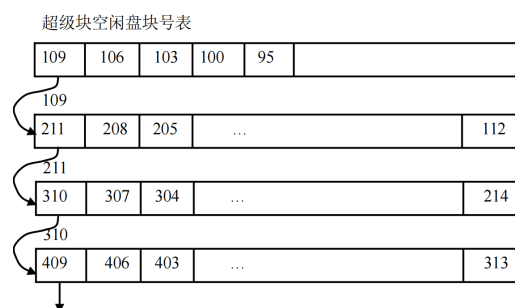


图 10: 成组链接法示意图

概念理解:

- 超级块: 它是整个文件系统的总指挥官, 它固定在磁盘分区的最开头 (比如 0 号或 1 号块), 永远不动。它记录了整个分区的基本信息
- 超级块空闲盘块号表: 超级块中有一个专门的区域, 用来存放一组空闲盘块的块号, 这个区域就叫做“超级块空闲盘块号表”。它就像一个小型的目录, 告诉系统哪些盘块是空闲的, 可以用来存储新文件或数据。

2.2.2 接口设计

文件描述符作为凭证, Open 方法建立连接, Close 方法断开连接。

1. 文件描述符 (File Descriptor, fd)

- 定义: 一个非负整数 (如 3, 4, 5...)。
- 作用: 它是用户进程与内核文件结构之间的抽象句柄。用户不需要接触底层的 Inode 或物理地址, 只需持有这个“号码牌”即可操作文件。
- 本质: 它是当前进程“打开文件表”数组的下标索引。

你的进程里有一个指针数组 `files[ ]`, 当你 open 一个文件时, 内核找一个空闲的位置 (比如下标 3), 让 `files[3]` 指向内核里的一个打开文件对象

然后函数返回 3 给你的程序, 下次你调用 `read(3, ...)`, 操作系统直接去查 `files[3]`, 顺藤摸瓜找到文件。

3 就是 `fd`。

2. Open 方法

- 功能: 打开指定路径的文件, 建立读写会话。
- 原型: `int Open(char *path, int mode);`
- 内部流程:
 - (a) 寻址 (**Path Lookup**): 根据路径查找目录, 找到文件的静态 `Inode` (身份证)。
 - (b) 安检 (**Check**): 检查用户权限 (读/写) 是否合法。
 - (c) 建档 (**State Creation**): 在内核中创建一个动态的 `file` 结构体, 初始化读写指针 (`offset`) 为 0。
 - (d) 发牌 (**Allocation**): 在进程的文件描述符表中找到一个空闲位置 (如下标 3), 指向上述结构体, 并将该下标作为 `fd` 返回。

3. Close 方法

- 功能: 关闭文件, 释放资源。
- 原型: `int Close(int fd);`
- 内部流程:
 - (a) 回收号码牌: 清空进程文件描述符表中 `fd` 对应的项, 使其可以被再次分配。
 - (b) 销毁档案: 减少内核中对应 `file` 结构体的引用计数。若计数归零, 则释放该结构体占用的内存。
 - (c) 释放关联: 断开与底层 `Inode` 的动态连接。

2.2.3 连续分配

连续分配方法要求每个文件在磁盘上占有一组连续的块

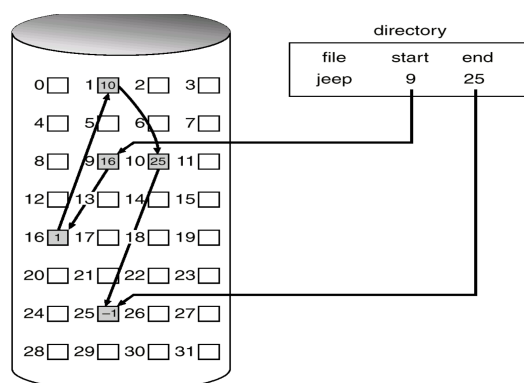


图 11: 链接分配示意图

- 特点：顺序访问容易且速度快，目录中文件存储位置信息简单；但容易产生碎片，需要定期对磁盘空间进行整理
- 存在的问题：为新文件找空间比较困难；文件很难增长

2.2.4 链接分配

以扇区为单位的链接分配/以区段（或簇）为单位的链接分配

- 优点：简单一只需起始位置；文件创建与增长容易
- 缺点：不能随机访问；块与块之间的链接指针需要占用空间；存在可靠性问题，如指针损坏

文件分配表 (**FAT**) 就是一种典型的以扇区为单位的链接分配方法

该表整个文件系统仅设一张，其结构如图所示。表的序号是物理块号，从 0 开始直至 $N - 1$ (N 为盘块总数)

FAT 表大小 = 磁盘块总数 $\times$ 每个表项的大小

对于 磁盘块总数 就是磁盘空间/ 磁盘块大小

对于 每个表项的大小 盘块越多，编号就越长，记录这个编号所需的空间就越大, 因为 1 字节 (Byte) = 8 位 (bit) 所以计算方法是：

每个表项的大小 (字节) = $\lceil \log_2(\text{磁盘块总数})/8 \rceil$

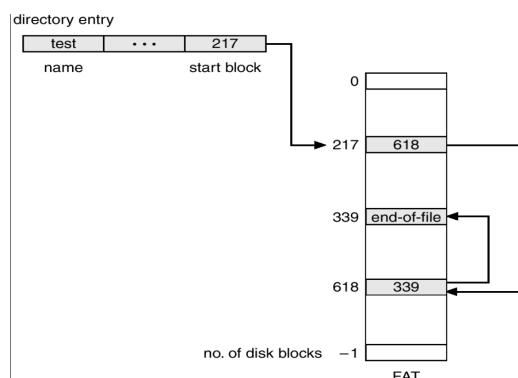


图 12: FAT 文件分配表示意图

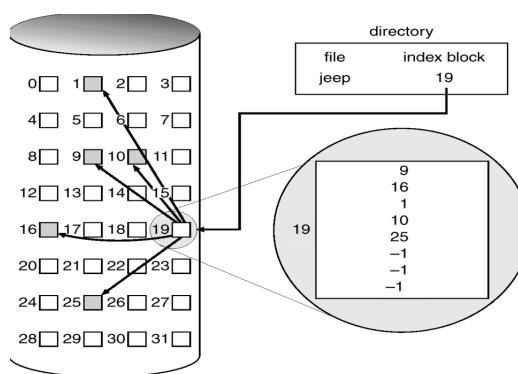


图 13: 索引分配示意图

2.2.5 索引分配

索引分配概念：为每个文件建立一个索引块，索引块中存放该文件所有数据块的地址，实现随机访问

- 优点：索引分配方法支持直接访问，不会产生外部碎片
- 缺点：但索引块要占用一定的存储空间，存取文件需要两次访问外存

example: 假设有一个文件 ABC.txt，内容分散在磁盘的第 8, 2, 5 号块中，目录项 (FCB)：记录文件名 ABC.txt，并记录索引块的地址，索引块 (第 100 号)：里面写着三个数字：8, 2, 5。用户想读文件的第 3 部分，系统找到第 100 号块（索引块），直接读取数组的第 3 项 → 发现是 5，磁头直接飞到 5 号块读取数据。

地址：

假设盘块大小为 4KB，一个盘块号占 4 字节

1. 直接地址：索引节点（Inode）中直接存放数据块的物理盘块号。

直接地址根本就没有给你“分配一个专门的块”来存钥匙，它只是在 Inode 里借了几个位置而已

寻址范围：通常假设 Inode 含 10 个直接地址项，则范围为 $10 \times 4KB = 40KB$ 。

2. 一次间接地址：索引节点指向一个索引块，该块中存放指向数据块的地址（即存放 $4KB/4B = 1024$ 个地址）。

寻址范围： $1024 \times 4KB = 4MB$ 。

3. 二次间接地址：索引节点指向一个一级索引块，一级块指向 1024 个二级索引块，二级块再指向数据块。

寻址范围： $1024 \times 1024 \times 4KB = 1M \times 4KB = 4GB$ 。

2.3 文件目录与路径

2.3.1 文件链接

概念：允许一个文件拥有多个名字，或者让一个文件指向另一个文件。这让用户和程序可以从不同的路径访问同一个文件，极大提高了文件组织的灵活性

1. 硬链接 (**Hard Link**)：硬链接是指多个不同的目录项（文件名）直接指向同一个索引节点（Inode）

采取的措施：引用计数机制（少一个链接，计数减一，计数为 0 时删除文件）

限制：不能跨文件系统、不能链接目录

2. 软链接 / 符号链接 (**Symbolic Link**)：软链接是一个独立的文件，包含指向另一个文件路径的文本字符串。不同文件有不同的 Inode

优点：可以跨文件系统、可以链接目录

3. 比较：为什么硬链接不能链接目录？

假设你有一个目录 /A/B，如果你在 B 下面创建了一个指向 A 的硬链接 `hard_link_to_A`

结果：路径变成了 /A/B/hard_link_to_A/B/hard_link_to_A/... 这就形成了一个环路 (Cycle) 。

2.4 文件操作接口

挂载：将外部文件系统的根目录连接到已有目录树的某个节点。

实现步骤：

- 1. 读取超级块：验证文件系统类型 (超级快: 文件系统大小、块大小、空闲 Inode 数、空闲块数、魔数 (标识文件系统类型))
- 2. 连接根目录：挂载点 dentry → 新文件系统根 Inode
- 3. 标记挂载点：d_mounted 标志

2.5 典型文件系统

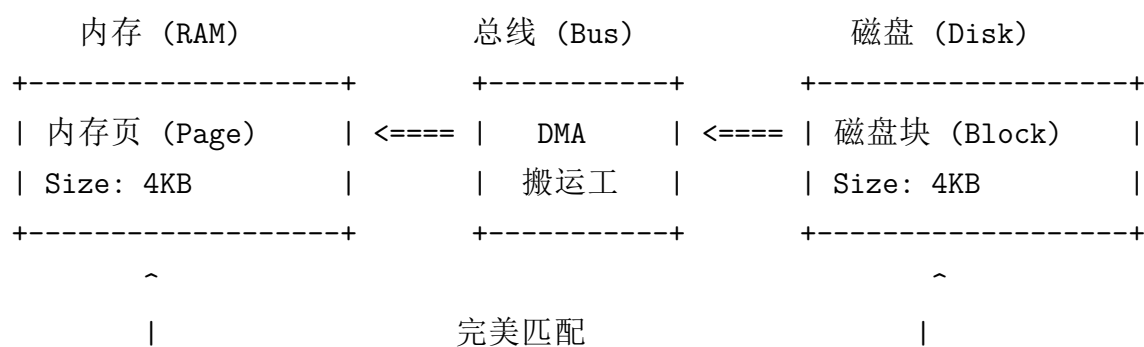
表 2: UNIX 与 FAT32 文件系统特性对比

特性	UNIX 文件系统	FAT32 文件系统
元数据位置	Inode (与文件名分离)	目录项 (包含所有信息)
物理分配方式	多级索引分配 (树状)	显式链接分配 (链式)
单文件上限	TB 级 (几乎无限制)	4GB (受限于 32 位 size 字段)
长文件名支持	原生支持	需 VFAT 扩展支持
可靠性	高 (引用计数、日志等)	中 (主要依靠 FAT 表备份)
实现复杂度	高	低
典型应用场景	Linux/macOS、服务器	U 盘、SD 卡、嵌入式设备

2.6 文件系统性能

1. 缓存技术 (**Caching**): 文件系统之所以需要“缓存技术”，核心原因只有一个：磁盘太慢了，而内存和 CPU 太快了。缓存技术就是把最近访问的数据保留在内存中，避免频繁访问慢速磁盘，从而提升整体性能。

- 缓冲区缓存 (Buffer Cache): 缓存磁盘块，减少物理读写次数 (单位：磁盘块 (4KB))
- 页面缓存 (Page Cache): 缓存文件数据页，提高文件读写效率 (单位：内存页 (4KB))



2. 预读与写回 (**Read-Ahead and Write-Back**): 原理：检测顺序访问，提前读取后续块
初始 4-8 块，连续命中：则窗口扩大，随机访问：则窗口缩小
3. 延迟写 (**Delayed Write**): 将写操作先缓存在内存中，待条件满足时再批量写入磁盘，减少磁盘写入次数，提高效率

2.7 虚拟文件系统 (VFS)

VFS 是一层软件抽象层，它位于应用程序和具体的文件系统实现之间，提供统一的文件系统接口

设计原因：Linux 支持的文件系统非常多（ext4, xfs, fat32, nfs...），它们的底层实现各不相同。FAT32 使用文件分配表，没有 Inode，Ext4 使用 Inode 和多级索引。如果程序员写代码时，读 FAT32 文件要用 `read_fat()`，读 Ext4 要用 `read_ext4()`，那简直是噩梦。

VFS 定义了一套统一的标准接口（POSIX 标准，如 `open`, `read`, `write`）。应用程序只管调用这些标准接口，VFS 负责把它们“翻译”成底层文件系统能听懂指令

作用：统一接口，屏蔽底层差异

四大核心数据结构：

- 超级块 (**super_block**)：表示一个挂载的文件系统，包含文件系统类型、大小、状态等信息
- 索引节点 (**inode**)：表示文件的元数据，如权限、所有者、时间戳、数据块位置等
- 目录项 (**dentry**)：表示目录中的一个文件或子目录，包含文件名和对应的 inode 号
- 文件对象 (**file**)：表示一个打开的文件，包含读写指针、访问模式等信息

有很多人会弄混 Inode 和 fd, 这里给出一个例子进行说明：

比如描述一个看电影的事情,这个电影存放在磁盘中,我打开是由 Inode 定位到这个”静态资源”，但是我现在有两个人观看，打开两次分配了两个 fd (1,2)，作为文件描述符，比如一个人 a 看了 10 分钟，一个人 b 看了 1 分钟，a 如果暂停后再看会调用 fd=1 回到 10min 处而不是 1 分钟。

3 设备管理

3.1 基本概念

I/O 设备及其分类：I/O 设备（Input/Output Devices，输入/输出设备）是计算机系统中除了 CPU（大脑）和内存（短期记忆）以外的所有外部设备的总称

1. 按使用特性:

- 人机交互设备: 键盘、鼠标、显示器
- 存储设备: 硬盘、U 盘、光驱
- 网络通信设备: 网卡、调制解调器

2. 按传输速率:

- 低速: 键盘 (10B/s)、鼠标 (100B/s)
- 中速: 打印机 (100KB/s)
- 高速: 磁盘 (100MB/s)、网卡 (1GB/s)

3. 按信息交换单位:

- 块设备: 硬盘、U 盘 (可随机访问)
- 字符设备: 键盘、串口 (流式访问)
- 网络设备: 网卡 (数据包)

I/O 端口: 要了解 I/O 端口, 首先要明白什么是设备控制器。设备控制器就像是设备的“翻译官”, 它负责把计算机发出的数字信号转换成设备能理解的形式, 反之亦然。

一个 I/O 设备就是通常由机械部件 (物理设备) 和电子部件 (控制器) 组成的。I/O 端口就是设备控制器与 CPU 或内存之间进行数据交换的接口

端口类型	作用
数据端口 (Data)	存放输入/输出数据 (缓冲)
状态端口 (Status)	指示设备忙/闲、错误 (握手)
控制端口 (Ctrl)	接收启动、复位等命令

I/O 控制方式: 解决“超快的 CPU”和“超慢的 I/O 设备”之间的速度矛盾, 尽可能把 CPU 从繁重的 I/O 搬运工作中“解放”出来

1. 程序控制方式 (Programmed I/O): CPU 不断地去查看设备控制器的状态寄存器。CPU 问: “你好了吗?”, 设备: “没”。CPU 再问: “好了吗?”, 设备: “没”……直到设备说“好了”, CPU 才去读写数据

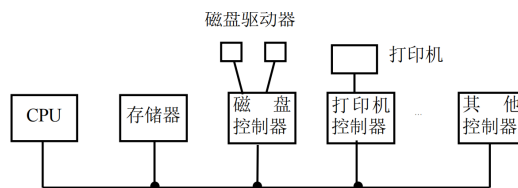


图 14: 总线示意图

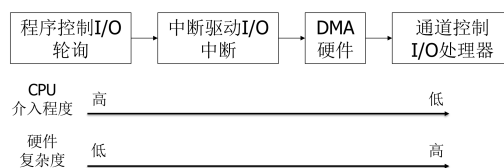


图 15: I/O 控制方式示意图

- 优点：实现简单，适用于高速设备
 - 缺点：CPU 忙于轮询，效率低下
2. 中断驱动方式 (Interrupt-Driven I/O): CPU 发出指令让设备开始工作，然后 CPU 转头去干别的事。当设备做完了，会发一个中断信号（就像敲门或电话）告诉 CPU。CPU 停下手头工作，去处理数据
- 优点：提高 CPU 利用率，适用于中速设备
 - 缺点：每次中断都需要 CPU 介入，开销较大
 - 适用：键盘、鼠标等低中速设备
3. DMA 方式 (Direct Memory Access): DMA 控制器是一个专门的搬运工，CPU 只需告诉它“去把数据从设备搬到内存的哪个位置”，然后就可以继续干别的事了。DMA 控制器自己完成搬运任务，发送中断通知 CPU
- 优点：大幅减少 CPU 介入，适合大批量数据传输
 - 适用：磁盘网卡等高速设备
4. 通道控制方式 (Channel I/O): 通道控制器是一个更高级的搬运工，它不仅能搬运数据，还能执行一系列复杂的 I/O 操作指令。CPU 给通

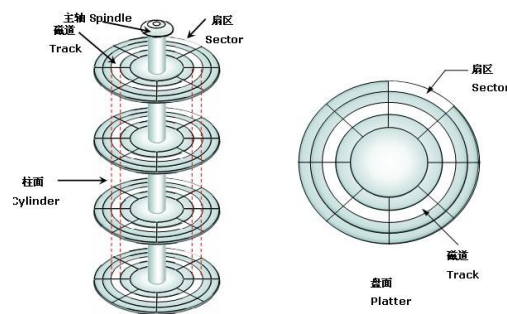


图 16: 机械硬盘结构示意图

道发送一段“通道程序”（任务清单）。通道自主执行多个 I/O 操作，统筹管理多台设备，甚至可以进行简单的逻辑处理

- 优点：极大减轻 CPU 负担，适用于大型机
- 适用：大型机系统

3.2 硬盘

3.2.1 机械硬盘

机械硬盘 (HDD) 是一种传统的存储设备，利用磁性材料在旋转的盘片上存储数据。

总访问时间 = 寻道时间 + 旋转延迟 + 传输时间

性能瓶颈：机械运动，寻道和旋转导致访问延迟高

3.2.2 固态硬盘

SSD 内部没有任何机械运动部件，它更像是一个巨大的、断电不丢数据的内存条 组成：

- NAND 闪存芯片：存储数据的主要介质
- 控制器：SSD 的“大脑”，负责管理数据读写和内部算法
- DRAM 缓存：提高读写速度

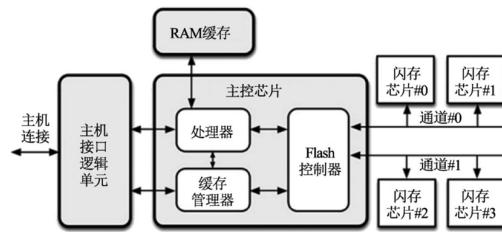


图 17: 固态硬盘结构示意图

特性:

- 读写单位是“页” (Page): 你可以一次只读或写 4KB 的数据
- 擦除单位是“块” (Block): 你不能只删除某一个页, 要删就得把整个块 (几百个页) 全部擦除
- 写前必须擦除 (Erase before Write)

SSD 中还有一个非常核心的概念——**FTL (Flash Translation Layer, 闪存转换层)**

FTL 是固态硬盘 (SSD) 控制器内部的一个核心软件层, 它的主要作用是解决操作系统 (基于逻辑块地址 LBA) 与 NAND Flash 物理特性 (基于物理页地址 PPA, 且写前必须擦除) 之间的矛盾

三大功能和一大问题:

1. 地址映射 (Address Mapping): 将逻辑块地址 (LBA) 映射到物理页地址 (PPA)
2. 垃圾回收 (Garbage Collection): 回收无效数据页, 整理存储空间
3. 磨损均衡 (Wear Leveling): 均匀分配写入次数, 延长 NAND Flash 寿命
4. 问题: 写放大效应 (Write Amplification): 实际写入的数据量大于请求写入的数据量, 影响性能和寿命

机械硬盘 (HDD) 与固态硬盘 (SSD) 对比:

内存 (RAM) 与磁盘 (Disk/Storage) 对比:

对比维度	机械硬盘 (HDD)	固态硬盘 (SSD)	核心差异点
存储介质	磁性盘片 (Magnetic Platter)	NAND Flash 芯片	物理原理不同 (磁 vs 电)
内部结构	机械结构: 马达、盘片、磁头臂	纯电子结构: 控制器、Flash 芯片	HDD 有活动部件, SSD 无
随机访问速度	慢 (~10 ms)	极快 (~0.1 ms)	SSD 快约 100 倍 (核心优势)
顺序读写速度	一般 (100–200 MB/s)	极快 (500–3500 MB/s)	SSD 拷贝大文件更快
抗震性	差 (怕摔, 有机械部件)	优秀 (抗震, 无机械部件)	SSD 更适合笔记本/移动设备
功耗与噪音	高 (5–10W), 有马达噪音	低 (2–4W), 完全静音	SSD 更省电、安静
使用寿命	物理磨损 (寿命较长)	P/E 擦写次数有限	SSD 写多了会坏 (但家用足够)
价格成本	低 (适合存大量冷数据)	高 (适合装系统和软件)	HDD 单位容量更便宜

3.2.3 磁盘调度算法

假设当前磁头位置为: 53

请求队列: 98, 183, 37, 122, 14, 124, 65, 67

1. 先来先服务 (FCFS):

53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67

总寻道距离 = $|53-98| + |98-183| + |183-37| + |37-122| + |122-14| + |14-124| + |124-65| + |65-67| = 640$

2. 最短寻道时间优先 (SSTF): 找和当前磁头位置距离最近的请求

53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183

3. 扫描算法 (SCAN): 磁头会沿着一个方向 (比如从小到大) 一直移动, 处理沿途的所有请求, 直到碰到磁盘的物理边界 (0 号磁道或最大磁

对比维度	内存 (Memory / RAM)	磁盘 (Disk / Storage)	通俗比喻
技术类型	DRAM (动态随机存取存储器)	HDD (磁性) 或 SSD (闪存)	工位桌面 vs 仓库书架
系统角色	CPU 的“工作台”，数据必须先加载到这里 CPU 才能处理	数据的“仓库”，负责长期保存文件和数据	桌上正在看的书 vs 书架上放着的书
速度差异	最快 (纳秒级)，比 SSD 还要快几十上百倍	较慢 (毫秒/微秒级)，即使是 SSD 也比内存慢得多	伸手拿文件 vs 走路去仓库取
断电保存	易失性 (Volatile)，断电/关机数据全丢	非易失性 (Non-Volatile)，断电后数据依然保留	下班清空桌面 vs 存入档案柜
典型容量	小 (8GB-64GB)	大 (512GB-10TB+)	桌面空间有限 vs 仓库很大
解决卡顿	解决多任务卡顿、程序无响应 (增大桌面，能同时干更多事)	解决开机慢、加载慢 (提升从仓库取货的速度)	针对不同瓶颈：RAM 提升并行，磁盘升级提升存取

道号)，才掉头往回走

53 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → 14 → 37

4. **LOOK** 算法: 磁头沿一个方向移动，处理沿途的请求，但不会跑到磁盘边界，而是在没有请求的地方就掉头 53 → 65 → 67 → 98 → 122 → 124 → 183 → 14 → 37
5. 循环扫描算法 (**C-SCAN**): 磁头沿一个方向移动，处理沿途的请求，跑到磁盘边界后直接跳回另一端，继续处理请求 53 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → 0 → 14 → 37
6. 循环 **LOOK** 算法 (**C-LOOK**): 磁头沿一个方向移动，处理沿途的请求，跑到最后一个请求后直接跳回最前面的请求，继续处理 53 → 65 → 67 → 98 → 122 → 124 → 183 → 14 → 37

7. **N-Step-SCAN** 算法: 将请求队列分成若干个长度为 N 的小队列, 磁头每次只处理一个小队列, 处理完后再处理下一个小队列

示例路径(假设 $N = 4$): 我们将请求队列 {98, 183, 37, 122, 14, 124, 65, 67} 分为两组:

- 子队列 1: {98, 183, 37, 122}
- 子队列 2: {14, 124, 65, 67}

第一步 (处理子队列 1): 从 53 开始, 按 SCAN (向大方向): $53 \rightarrow 98 \rightarrow 122 \rightarrow 183 \rightarrow (\text{掉头}) \rightarrow 37$

第二步 (处理子队列 2): 从 37 开始, 按 SCAN (接力方向): $37 \rightarrow 14 \rightarrow (\text{掉头}) \rightarrow 65 \rightarrow 67 \rightarrow 124$

完整路径: $53 \rightarrow 98 \rightarrow 122 \rightarrow 183 \rightarrow 37 \rightarrow 14 \rightarrow 65 \rightarrow 67 \rightarrow 124$

8. **FSCAN** 算法: 核心逻辑: FSCAN 是 N-Step-SCAN 的简化版, 它只使用 ** 两个子队列 **。

- ** 当前队列 **: 包含当前所有需要处理的请求, 由磁盘调度按 **SCAN** 算法进行处理。
- ** 缓冲队列 **: 在扫描期间新出现的所有请求, 都会被放入这个缓冲队列, 暂不处理。
- 当“当前队列”处理完后, 系统才会切换去处理“缓冲队列”。这能有效防止磁臂粘着, 保证所有请求最终都能被处理。

示例路径: 由于本题给出的请求队列是 ** 静态 ** 的 (即假设所有请求在开始时都已存在, 且期间没有新请求插入), FSCAN 的行为将与标准的 SCAN 算法一致, 因为它会把这 8 个请求全部视为“当前队列”并锁定。

$53 \rightarrow 65 \rightarrow 67 \rightarrow 98 \rightarrow 122 \rightarrow 124 \rightarrow 183 \rightarrow 199 \rightarrow 14 \rightarrow 37$

注: 如果处理过程中有新请求 (如 55) 到达, 它不会被立即插队处理, 而是会留到下一轮扫描。

3.2.4 分区与格式化

分区：

1. MBR (Master Boot Record):

- 最大支持 2TB 磁盘
- 最多支持 4 个主分区
- 分区表存储在磁盘的第一个扇区（512 字节）

2. GPT (GUID Partition Table):

- 支持超过 2TB 的大容量磁盘
- 支持最多 128 个分区
- 分区表存储在磁盘的多个位置，提供冗余备份

格式化：

分完区后，房间（分区）虽然有了，但里面还是空的，没有货架，也没有索引，货物扔进去就找不到了。格式化就是为了解决数据的管理问题。它分为两类

1. 低级格式化 (Low-Level Formatting):

这是物理层面的操作。它在磁盘的磁性表面上划分磁道（Track）和扇区（Sector），并打上电子标记

2. 高级格式化 (High-Level Formatting):

建立文件系统

3.3 时钟

时钟四大作用：

1. 维护系统时间（当前时间）
2. 驱动进程调度（时间片）

3. 实现定时功能 (sleep、timeout)

4. 统计 CPU 使用时间

硬件定时器和软定时器：

没有硬件定时器（挂钟）提供时间基准，软定时器（日程表）就无法运作

硬件定时器就像墙上的挂钟。它只负责机械地“滴答、滴答”走字，客观且永不停歇。软定时器就像你的日程表。你看着墙上的挂钟（硬件定时器），发现到了 9:00，于是去开会；发现到了 12:00，于是去吃饭

表 3: 硬件定时器与软定时器对比

特性	硬件定时器 (Hardware Timer)	软定时器 (Software Timer)
本质	物理设备 (电路/芯片)	内核功能 (数据结构/代码)
来源	由晶振、分频器和计数器芯片组成	基于硬件中断，由操作系统内核模拟
功能	产生周期性的物理中断信号 (Tick)	在特定时间点触发回调函数 (Function)
精度	极高，由晶振频率决定	受限于系统中断频率 (HZ) 和系统负载
角色	系统的”脉搏”	应用程序的”闹钟”

时钟中断：硬件定时器每隔固定时间产生中断

时钟中断就是硬件定时器每隔几毫秒给 CPU 打的一个“骚扰电话”。CPU 接起电话（响应中断），迅速地更新系统时间、检查闹钟，并看看当前进程是不是该下台了，从而维持整个操作系统的秩序

硬件链路：晶振（产生震荡）→ 分频器（降频）→ 定时器 → 中断控制器 → CPU

动作：硬件定时器会按照预设的时间间隔（比如每 1ms），无条件地给 CPU 发送一个中断信号，强迫 CPU 停下手头的活来处理这个信号

3.4 I/O 系统层次架构

1. 硬件设备层：包括各种物理 I/O 设备，如磁盘、键盘、显示器等。
2. 中断处理程序层：负责响应硬件设备发出的中断请求，通知操作系统有 I/O 事件发生。

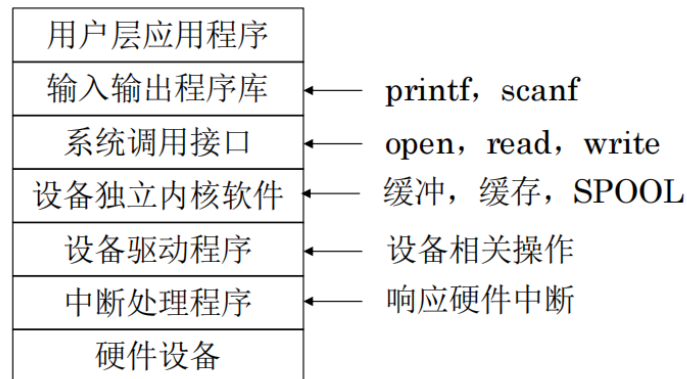


图 18: I/O 系统层次架构示意图

3. 设备驱动程序层：为每种具体的 I/O 设备提供操作接口，封装设备的硬件细节。
4. 设备独立内核软件层：提供统一的 I/O 接口，屏蔽不同设备驱动的差异。
5. 系统调用接口层：为用户进程提供标准的 I/O 操作接口，如 `read`、`write` 等。
6. 输入输出程序库层：为应用程序提供更高级的 I/O 功能封装，如文件操作库。

3.5 中断处理程序

3.5.1 上下文保护与恢复

保存内容：

- 程序计数器 (PC / IP)：记录了“被打断时执行到了哪行代码”（存盘点）
- 程序状态字 (PSW / EFLAGS)：记录了当前的运算状态（比如是否进位、是否为零、当前权限等级）
- 通用寄存器 (EAX, EBX, ECX...)：记录了程序正在计算的中间变量

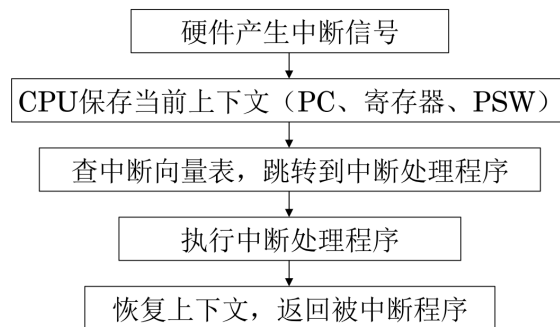


图 19: 中断处理程序示意图

- 栈指针 (SP): 记录了当前函数调用栈的位置

保存位置: 栈空间 (内核栈)

保存方式:

1. 硬件自动保存: CPU 在响应中断时, 自动将 PC 和 PSW 压入内核栈
2. 软件手动保存: 中断处理程序开始时, 手动将通用寄存器和 SP 压入内核栈

中断嵌套: 处理中断 A 时, 又发生中断 B, 如果 B 优先级高, 可以打断 A

嵌套造成的问题:

- 栈溢出风险: 每次嵌套都要保存上下文
- 实时性影响: 低优先级长时间得不到服务
- 复杂性增加: 临界区保护更复杂

3.5.2 上半部与下半部机制

为什么要分离:

如果不分两半: CPU 一口气把所有事情做完 (比如处理网络包、写硬盘)。这会耗时很久, 导致 CPU 长时间“失聪”, 丢失其他重要中断, 系统卡顿

- 中断处理应尽可能快
- 某些工作可以延迟
- 避免在中断上下文做复杂操作

于是我们分为上下两部分

表 4: 上半部与下半部机制对比

特性	上半部 (Top Half)	下半部 (Bottom Half)
别名	硬中断 (Hardirq)	软中断 (Softirq) / 延迟处理
执行时机	中断发生后立刻执行	稍后执行 (系统空闲或中断退出前)
中断状态	关闭中断 (CPU 听不到别的)	开启中断 (CPU 可以响应新中断)
主要任务	1. 签收硬件信号 (Ack) 2. 拷贝少量数据 3. 登记下半部 (告诉 OS 有活要干)	1. 复杂的数据处理 2. 协议栈解析 3. 磁盘 I/O 操作
耗时要求	极快 (微秒级)	可以较长
能否睡眠	绝对禁止 (会导致死锁)	Workqueue 方式可以睡眠

3.6 设备独立内核软件技术

缓冲区 (Buffer) 是一块临时存储区域, 用于在数据传输过程中暂时存放数据, 以弥补生产者和消费者之间的速度差异

举个例子: CPU 是“法拉利”, I/O 设备 (如打印机、硬盘) 是“老牛车”, 如果没有缓冲, 法拉利跑一米就得等老牛车走一米。有了缓冲, CPU 可以把一堆数据扔进缓冲区 (仓库), 然后去干别的, 让 I/O 设备慢慢搬。如果没有缓冲, 每传输 1 个字节就要打断一次 CPU, 有了缓冲, 凑满 4KB (一个缓冲区大小) 才打断一次 CPU, 效率大增

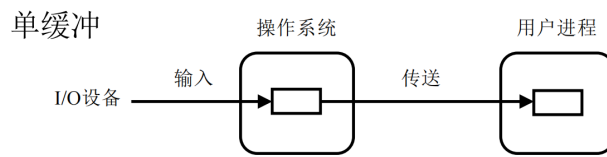


图 20: 单缓冲示意图

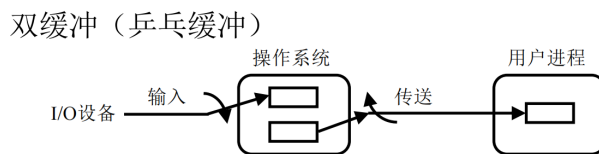


图 21: 双缓冲示意图

3.6.1 缓冲技术

单缓冲:

- 输入阶段: 设备把数据写进缓冲区 (此时 CPU 不能动这个缓冲区, 必须等待)。
- 传送阶段: 缓冲区满了, 操作系统把数据从缓冲区搬到用户区。
- 算阶段: CPU 开始处理用户区的数据。

计算公式: 假设: $M =$, $C = CPU$ 处理一块数据的时间 $= M + C$
设备和 CPU 不能同时工作

双缓冲:

- 设备往 1 号缓冲区倒数据。
- CPU 同时从 2 号缓冲区取数据。
- 当 1 号满了、2 号空了, 他俩交换目标。设备写 2 号, CPU 读 1 号。

计算公式: $\max(C, M)$ 设备和 CPU 可以同时工作

环形缓冲:

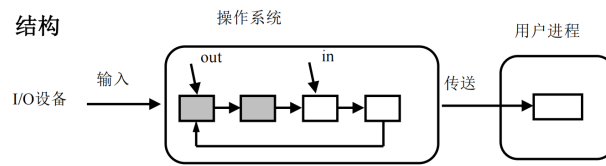


图 22: 环形缓冲示意图

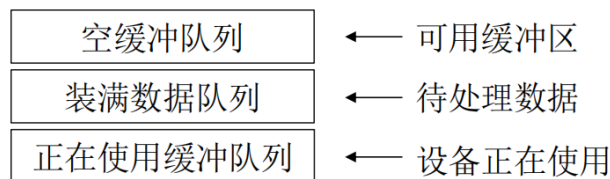


图 23: 缓冲池示意图

输入指针 (In): 指向下一个空缓冲区 (给设备装货用) 输出指针 (Out): 指向下一个满缓冲区 (给 CPU 卸货用)

空: $out == in$ 满: $(in + 1)$

- 充分利用缓冲区
- 适合流式数据 (音频、串口)

缓冲池:

缓冲池由多个缓冲区组成, 每个缓冲区都有一个缓冲首部 (记录身份信息, 如设备号、状态) 和缓冲体 (真正存数据的地方) 灵活高效

- 空闲缓冲队列: 这里面全是空的缓冲区, 谁想用都可以来拿
- 输入队列: 这里面全是装满了数据的缓冲区。这些数据是从设备读进来的, 正在排队等着 CPU (用户进程) 来取
- 输出队列: 这里面也全是装满了数据的缓冲区。但这些数据是 CPU 算出来的, 正在排队等着输出到设备上去

工作流程:

1. 设备从空队列获取缓冲区

- 2. 填充数据后放入装满队列
- 3. CPU 从装满队列取出处理
- 4. 处理完放回空队列

缓冲缓存的对比：

表 5: 缓冲与缓存对比		
维度	缓冲 (Buffer)	缓存 (Cache)
英文	Buffer	Cache
核心目的	平滑速度差异 (协调快慢)	加速数据访问 (减少延迟)
数据使用	通常一次	可能多次
方向	单向 (设备到内存或内存到设备)	双向 (CPU 与内存之间)
出现时机	数据在传输/移动过程中	数据在反复使用过程中
典型策略	FIFO (先进先出), 满了就送走	LRU (最近最少使用), 热点保留, 冷门淘汰
生活比喻	卸货区的传送带	图书馆的热门书架

3.6.2 异步 I/O (AIO)

3.6.3 异步 I/O (AIO)

异步 I/O 允许程序发起 I/O 请求后立即返回，无需等待 I/O 完成，从而提高系统的并发处理能力。

工作流程 (以 `aio_read()` 函数为例)：

- 1. 发起请求：程序调用异步 I/O 函数（如 `aio_read()`），立即返回，不阻塞。
- 2. 继续执行：程序继续执行后续代码，处理其他任务。
- 3. I/O 完成：操作系统完成 I/O 操作后，通过信号或回调函数通知程序。

表 6: 同步 I/O 与异步 I/O 对比

维度	同步阻塞 I/O (Blocking I/O)	异步 I/O (Asynchronous I/O)
调用后行为	线程被挂起 (Blocked), 必须等待 I/O 完成	函数立即返回, 线程继续执行其他任务
CPU 状态	空闲等待, 切换到其他进程	持续工作, 执行后续代码
通知机制	I/O 完成后线程被唤醒	通过信号 (Signal) 或回调函数 (Callback) 通知
资源消耗	高并发时需要大量线程 (每个连接一个线程)	少量线程即可处理大量并发请求
编程复杂度	简单直观 (顺序执行)	较复杂 (需要回调处理、状态管理)
典型函数	<code>read()</code> , <code>write()</code>	<code>aio_read()</code> , <code>aio_write()</code>
生活比喻	站在柜台前傻等面做好	拿取餐器离开, 面好了震动通知
适用场景	简单、低并发应用	高并发服务器 (Web、数据库)

4. 处理结果：程序在回调函数中处理 I/O 结果。

优缺点：

- 优点：
 - 提高 CPU 利用率
 - 适合 I/O 密集型应用
- 缺点：——> 非重点
 - 编程模型复杂，需要处理回调和状态
 - 调试困难，错误不易追踪
 - 不是所有操作系统都完全支持

3.6.4 SPOOLing 技术 (假脱机)

SPOOLing 是操作系统中为了解决“独占设备”（如打印机）无法被多个进程共享，以及“CPU 与外设速度差异过大”这两个核心矛盾而诞生的技术

SPOOLing 工作原理（以打印为例）：

场景对比：

- 没有 SPOOLing 时：

1. Word 申请占用打印机，打印机被独占。
2. Word 发送第 1 页 → 打印机打印（耗时约 5 秒）。
3. 在此期间，Word 界面卡死，CPU 必须等待，无法切换到其他任务。
4. 发送第 2 页... 直到 100 页全部打印完成，Word 才恢复响应，打印机才释放。
5. 其他进程（如 Excel）想打印？必须排队等待。

- 有了 SPOOLing 后：

1. Word 点击打印。
2. SPOOLing 程序立即将这 100 页数据快速保存到硬盘的”输出井”（缓冲区）中。
3. SPOOLing 告诉 Word：”打印任务已接收！”（实际未真正打印，只是存储）。
4. Word 瞬间恢复响应，用户可以继续编辑文档或执行其他操作。
5. 操作系统在后台由 SPOOLing 输出进程慢慢地将硬盘中的数据发送给打印机。
6. 其他进程的打印任务也可以同时提交到输出井，排队依次处理。

SPOOLing 系统组成：

1. 输入井 / 输出井：磁盘上的缓冲区，存放等待输入/输出的数据。

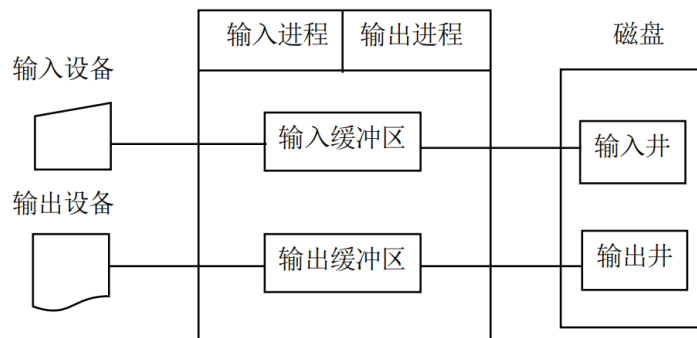


图 24: SPOOLing 系统结构示意图

2. 输入缓冲区 / 输出缓冲区：内存中的高速缓冲，加速数据传输。

3. 输入进程 / 输出进程：负责在井与缓冲区之间搬运数据。

4. 请求表：记录所有待处理的 I/O 请求及其状态。

优缺点：——> 非重点

优点：

- 提高设备利用率：将独占设备改造为虚拟的共享设备。
- 提高 CPU 效率：进程无需等待慢速设备，立即返回继续执行。
- 实现虚拟设备：一台物理打印机可以被多个进程“同时”使用（实际是排队）。

缺点：

- 需要占用额外的磁盘空间作为缓冲。
- 增加了系统复杂度，需要额外的进程调度和管理。

3.6.5 设备分配与回收

设备分类：

1. 独占设备：一次只能被一个进程使用（如打印机）

2. 共享设备：可同时被多个进程使用（如磁盘）
3. 虚拟设备：通过 SPOOLing 技术将独占设备虚拟化为共享设备

分配策略：

- 先请求先服务 (FCFS)：按照请求顺序分配设备
- 优先级调度：根据进程优先级分配设备
- 死锁避免：在分配设备时考虑避免死锁的发生

设备独立性：

设备独立性是指应用程序使用逻辑设备名来请求设备，由操作系统将逻辑设备名映射到物理设备名，从而使应用程序独立于具体的物理设备。

示例：应用程序请求”打印机”，操作系统将其映射到 /dev/lp0（具体的物理打印机）。

保护机制：

操作系统通过权限检查来保护设备资源，防止未授权访问。

在 Linux 系统中，设备文件具有与普通文件相同的权限机制：

- 读权限 (r)：允许从设备读取数据
- 写权限 (w)：允许向设备写入数据
- 执行权限 (x)：对设备文件通常无意义

示例：`crw-rw---- 1 root lp /dev/lp0`

- c：字符设备
- rw-：所有者（root）可读写
- rw-：组用户（lp 组）可读写
- ---：其他用户无权限